CITYJSON DYNAMIZER EXTENSION

Mapping and Encoding Specification

CityGML 3.0 Dynamizer module to CityJSON 2.0 Extension

Extension version 2.0.0

Normative source: OGC CityGML 3.0 Part 1

Encoding reference: OGC CityGML 3.0 Part 2

Target: CityJSON 2.0 Extension mechanism

Schema baseline: CityJSON 2.0.2

11 August 2026

This document is the technical specification accompanying dynamizer.ext.json version 2.0.0. It describes the mapping from the CityGML 3.0 Dynamizer module to CityJSON 2.0, the resulting extension structure, the rationale behind the main encoding decisions, and the limitations and validation requirements of the proposed approach. The JSON schema defines the structure used to validate extension files, while this report explains how that structure should be interpreted and applied.

Item	Value
Status	Research implementation and public release
Extension version	2.0.0
Compatible CityJSON version	2.0
CityJSON schema baseline	2.0.2
Source conceptual model	CityGML 3.0 Dynamizer module

Note: This work preserves the meaning of the supported CityGML Dynamizer concepts while adapting their representation to CityJSON. It does not reproduce the XML structure element by element. Some complex values, such as geometry and appearance changes, are not currently supported and may therefore not be fully preserved during conversion.

1. Purpose, scope, and deliverables

The CityGML Dynamizer module represents time-varying values for properties of city objects. CityJSON 2.0 has no native Dynamizer module, so the capability is introduced through the CityJSON Extension mechanism. The goal is to provide an encoding that follows CityJSON rules, preserves the meaning of the supported CityGML concepts, can be validated using existing tools, and supports embedded data, external files, and sensor services.

1.1. Included concepts

1. Dynamizer and the association to one target property.
2. AbstractTimeseries and AbstractAtomicTimeseries as non-instantiable modelling abstractions.
3. GenericTimeseries with typed scalar TimeValuePair records.
4. CompositeTimeseries with referenced TimeseriesComponent records.
5. StandardFileTimeseries and TabulatedFileTimeseries.
6. SensorConnection for remote or live observation sources.
7. TimePosition, exact instants, durations, units, and gml:CodeType adaptations.

1.2. Explicit exclusions

1. Changes over time to geometry, implicit geometry, and appearance are not currently supported.
2. The CityGML Versioning module is not covered by this extension.
3. When using CityJSONSeq, the target CityObject, its Dynamizer, and its time series may be stored in separate features. Additional packaging rules are therefore needed to keep these related objects together or to resolve references across features.

1.3. Package deliverables

Deliverable	Role
dynamizer.ext.json	Executable CityJSON Extension schema.
This specification	Concept mapping, encoding decisions, conformance, and limitations.
Five CityJSON examples	Temperature use case encoded through each supported source pattern.
CSV and TimeseriesML resources	Time-series data referenced by the external-file examples.
Additional scripts	Optional tools provided to generate the examples and perform additional validation checks. They are not part of the extension specification.

2. Normative baseline and conformance language

CityGML 3.0 Part 1 is the normative conceptual source. CityGML 3.0 Part 2 and the Dynamizer XSD describe the GML/XML encoding and clarify datatypes and multiplicities. XML technologies such as XPath, XLink, substitution groups, and namespaces are encoding mechanisms, not domain concepts that must be reproduced literally in JSON. The target rules are CityJSON 2.0 and its Extension mechanism, tested against the CityJSON 2.0.2 schemas.

Label	Meaning in this specification
Direct	The source concept has a close CityJSON representation with no important semantic change.
Adapted	The source meaning is retained, but the JSON organisation or lexical form changes.
Schema abstraction	A CityGML abstract class contributes shared constraints but has no instance.
Excluded	The concept is intentionally outside extension version 2.0.0.
Semantic rule	The rule cannot be fully established by the extension JSON Schema alone.

3. CityJSON constraints that shape the design

The mapping is constrained by CityJSON's data model. A new CityObject type defined by an extension begins with '+'. An attribute added to an existing core CityObject also begins with '+'. Properties inside a newly defined extension CityObject are owned by that schema and therefore do not need a prefix. New

CityObjects inherit the common CityObject members by referencing cityobjects.schema.json#/_AbstractCityObject.

CityJSON rule	Application in the Dynamizer Extension
New CityObject type names begin with +.	The extension defines +Dynamizer, +GenericTimeseries, +CompositeTimeseries, +StandardFileTimeseries, and +TabulatedFileTimeseries.
Attributes added to an existing CityJSON type begin with +.	The extension can add +dynamizer to a Building or another core CityObject. This optional attribute lists the Dynamizers that target that object.
Properties defined inside a new extension CityObject do not need +.	Properties such as target, attributeRef, and dynamicData belong to +Dynamizer, so they do not need the prefix.
All CityObjects follow a common basic structure.	Each new extension type reuses _AbstractCityObject, which provides common properties such as attributes, parents, children, and geographicalExtent. _AbstractCityObject is a schema definition, not an object added to the CityJSON file.
Parents and children describe hierarchical relationships.	They are not used for Dynamizer references because “changes a property” and “uses a time series” are not hierarchical relationships.

4. Hybrid architecture and object identity

The design is hybrid because it chooses representation according to identity and reuse rather than forcing every UML class into the same JSON structure. Concepts that must be found, referenced, reused, or recursively composed are independent CityObjects. Small records that only make sense inside their owner are embedded objects.

Concept	Representation	Design reason
Dynamizer	Independent +Dynamizer	Stable ID, generic target, direct query and inverse discovery.
Concrete time series	Independent extension CityObjects	Each time series has a stable ID, can be reused by multiple Dynamizers, and can be referenced by a composite time series.
SensorConnection	Embedded in +Dynamizer	Connection configuration is owned by one association.
TimeValuePair	Embedded array item	A timestamp/value record has no useful independent lifecycle.
TimeseriesComponent	Embedded array item	It describes how another identified series is used.
Abstract classes	Schema-level constraints	CityGML marks them non-instantiable.

4.1. Main references and optional reverse link

A +Dynamizer uses three references to identify the affected property and its source of dynamic values:

```
"target": "DEBY_LOD2_4959457",
"attributeRef": "/attributes/temperature",
"dynamicData": "series-temperature"
```

Together, target and attributeRef identify the affected property. The +dynamizer link is optional because the same relationship is already defined by target. When it is included, it must be consistent with the target value.

Member	Question answered	Meaning
target	Which CityObject is affected?	ID of the CityObject containing the property that changes over time. This is the main reference to the affected CityObject.
attributeRef	Which property inside the target is affected?	JSON Pointer identifying the specific property inside the target CityObject.
dynamicData	Which series supplies the values?	ID of the time-series CityObject that provides the changing values.
+dynamizer	Which Dynamizers point to this target?	Optional reverse link stored on the target CityObject. It lists the Dynamizers associated with that object and makes them easier to find.

Note: CityJSON parents and children primarily encode object hierarchy, for example Building to BuildingPart. A Dynamizer-to-target relationship means 'this object changes this property'; a series relationship means 'this object obtains values from that series'. Neither is a physical containment hierarchy. Using role-specific IDs avoids giving parents/children a second incompatible meaning.

5. Extension structure

5.1. New CityObject types

Type	CityGML source	Role
+Dynamizer	Dynamizer	Associates a target property with temporal data or a sensor.
+GenericTimeseries	GenericTimeseries	Stores typed timestamp/value pairs.
+CompositeTimeseries	CompositeTimeseries	Composes referenced series without copying observations.
+StandardFileTimeseries	StandardFileTimeseries	References a file governed by a time-series standard.
+TabulatedFileTimeseries	TabulatedFileTimeseries	References and describes a tabulated resource.

5.2. Embedded definitions

Definition	Purpose
Instant	Exact RFC 3339 date-time with explicit offset.
TimePosition	Exact compact instant or expanded GML-compatible temporal position.
Duration	XML Schema duration lexical form.
CodeValue	Compact code string or object preserving codeSpace.

SensorConnection	Connection metadata for observations and sensor descriptions.
TimeValuePair*	Five typed scalar timestamp/value record variants.
TimeseriesComponent	Series ID, repetitions, and optional additional gap.

6. Complete concept mapping

CityGML 3.0 concept	CityJSON 2.0 realisation	Status / rationale
Dynamizer	+Dynamizer	Adapted: independent object with explicit target.
dynamized feature association	target	Adapted: source containment becomes an ID.
attributeRef	attributeRef	Adapted: XPath becomes target-relative JSON Pointer.
dynamicData	dynamicData	Adapted: reference to concrete series CityObject.
sensorConnection	sensorConnection	Direct/adapted embedded definition.
AbstractTimeseries	shared properties	Schema abstraction; no instance.
AbstractAtomicTimeseries	shared atomic properties	Schema abstraction; no instance.
GenericTimeseries	+GenericTimeseries	Direct/adapted CityObject.
CompositeTimeseries	+CompositeTimeseries	Adapted referenced composition.
StandardFileTimeseries	+StandardFileTimeseries	Direct/adapted CityObject.
TabulatedFileTimeseries	+TabulatedFileTimeseries	Direct/adapted CityObject.
TimeValuePair	timeValuePair[] item	Embedded typed record.
TimeseriesComponent	component[] item	Embedded record referencing a series ID.
TimePosition	Instant or expanded object	Adapted lexical encoding.
duration	Duration string	XML Schema lexical family.
gml:CodeType	CodeValue	Adapted string/object dual form.
geometry/implicit/appearance values	No executable mapping	Excluded from version 2.0.0.
ADE hooks	No literal mapping	Intentional null mapping.

7. +Dynamizer property mapping

A +Dynamizer identifies a property whose value changes over time. The target member identifies the affected CityObject, while attributeRef identifies the specific property inside that object. The changing values may be provided by a time series referenced through dynamicData or obtained from a sensor

service described by sensorConnection. In accordance with the CityGML cardinalities, dynamicData and sensorConnection are optional in the extension schema. However, at least one data source is needed for the Dynamizer to provide changing values in practice.

CityGML property / source meaning	CityJSON member	Encoding and rule
Containing/dynamized feature	target	Required ID of the affected CityObject. This is the main reference from the Dynamizer to that object.
attributeRef	attributeRef	RFC 6901 JSON Pointer evaluated relative to target.
startTime	startTime	Optional TimePosition defining activation start.
endTime	endTime	Optional TimePosition defining activation end.
dynamicData	dynamicData	ID of a supported concrete Timeseries.
sensorConnection	sensorConnection	Embedded SensorConnection.
creationDate / terminationDate	same names	Feature lifecycle metadata.
validFrom / validTo	same names	Validity interval metadata.

Note: The original static value remains stored in the CityObject. For a given time, the application uses the corresponding value provided by the Dynamizer. If no dynamic value is available for that time, the application may use the static value as a fallback.

8. Why JSON Pointer is used

In the CityGML GML/XML encoding, attributeRef identifies a property in the XML structure. This expression cannot be copied directly to CityJSON because XML and JSON organize information differently. The extension therefore uses RFC 6901 JSON Pointer, a standard way of identifying a value inside a JSON object. A JSON Pointer is written as a sequence of property names separated by /.

Property to identify	attributeRef
Temperature attribute	/attributes/temperature
Unit stored inside temperature metadata	/attributes/temperatureMetadata/uom

Note: The extension schema verifies that attributeRef is a string beginning with /. An additional validation step must verify that target exists, that the pointer identifies an existing property inside that CityObject, and that the dynamic values are compatible with the selected property.

9. Temporal mapping

9.1. Exact instants

JSON has no native date-time datatype. Exact instants are therefore strings constrained by both format: date-time and a lexical pattern. RFC 3339 is used because it is the interoperable Internet profile of ISO 8601 date-time syntax and requires an explicit UTC marker or numeric offset in this extension.

```
"2026-08-11T10:30:00Z"
"2026-08-11T12:30:00+02:00"
```

9.2. Expanded TimePosition

gml:TimePositionType can carry less than a full instant and can express indeterminate positions or a temporal reference frame. A single RFC 3339 string would lose these facets. The extension therefore provides a compact exact-instant branch and an expanded object branch.

```
{
 "value": "2026-08",
 "indeterminatePosition": "after",
 "frame": "http://www.opengis.net/def/uom/ISO-8601/0/Gregorian",
 "calendarEraName": "Common Era"
}
```

Member	Purpose
value	Year, year-month, date, or date-time lexical value.
indeterminatePosition	now, before, after, or unknown.
frame	URI-reference identifying a temporal reference system.
calendarEraName	Optional calendar era label.

9.3. Duration

Duration is encoded as an XML Schema duration lexical string rather than a number. A number alone has no unit and cannot distinguish one month from a fixed number of seconds. The lexical family preserves years, months, days, time components, fractional seconds, and permitted negative durations. The week form P4W is correctly excluded because xs:duration does not use the alternative ISO week representation.

```
"P1D"
"PT30M"
"P1DT12H"
"-P120D"
```

10. Abstract and atomic time-series mapping

AbstractTimeseries and AbstractAtomicTimeseries organise the CityGML UML inheritance tree. They cannot appear as concrete instances. In JSON Schema, their effect is represented by repeating or referencing shared property schemas in concrete types. This preserves inherited semantics without inventing +AbstractTimeseries objects.

Source abstraction	Inherited meaning	Applied to
AbstractTimeseries	firstTimestamp, lastTimestamp	All four concrete series types.
AbstractAtomicTimeseries	observationProperty, uom	Generic, StandardFile, and TabulatedFile.

11. GenericTimeseries and typed TimeValuePair records

+GenericTimeseries stores observations directly in its attributes. valueType selects one mutually exclusive schema branch, and every item in timeValuePair must use the corresponding typed member. This makes the value type explicit and prevents ambiguous generic 'value' fields.

valueType	Required pair member	JSON datatype
int	intValue	integer
double	doubleValue	number
string	stringValue	string
uri	uriValue	URI-reference string
bool	boolValue	Boolean

```
{
 "type": "+GenericTimeseries",
 "attributes": {
  "observationProperty": "Temperature",
  "uom": "Cel",
  "valueType": "double",
  "timeValuePair": [
   {"timestamp": "2025-08-27T06:58:22.758Z", "doubleValue": 19.8}
  ]
 }
}
```

TimeValuePair remains embedded because its identity is its position within one series and its timestamp; it is not independently reused. JSON Schema checks the local type branch. Temporal ordering, duplicate timestamps, interval coverage, and interpolation remain semantic or application rules.

12. CompositeTimeseries and TimeseriesComponent

A +CompositeTimeseries defines an ordered sequence of components. Each TimeseriesComponent references another concrete series by ID, gives the repetition count, and may add a temporal gap. The observations are not copied, which preserves reuse and avoids duplication.

```
"component": [
 {"timeseries": "series-morning", "repetitions": 1},
 {"timeseries": "series-afternoon", "repetitions": 2, "additionalGap": "PT5M"}
]
```

Member	Encoding	Rule
timeseries	Non-empty ID string	Must resolve to a supported concrete series.
repetitions	Integer	A semantic profile should require a positive value.
additionalGap	Duration	Must be non-negative for this use.
component	Non-empty ordered array	Order determines composition sequence.

Note: A +CompositeTimeseries may include another composite time series. However, it must not eventually refer back to itself. For example, if series-a includes series-b, then series-b must not include series-a. An additional validation step is required to detect this situation.

13. File-based time series

13.1. StandardFileTimeseries

A standard-file series points to an external resource whose organisation is defined by an identified standard, such as TimeseriesML. The CityJSON file carries discovery and semantic metadata, while the observations remain in the resource.

Member	Meaning
fileLocation	Relative path or URI-reference to the resource.
fileType	CodeValue identifying the standard or format.
mimeType	Optional CodeValue describing the media type.
observationProperty	Observed phenomenon represented by the values.
uom	Optional unit of measure.

13.2. TabulatedFileTimeseries

A tabulated resource does not have a self-describing domain model, so the CityJSON object provides parsing and column-selection metadata. Column selectors may use a number or a name. The structural schema remains faithful to the CityGML cardinalities and does not force operational completeness that is absent from the source XSD.

Property group	Members	Operational interpretation
Parsing	numberOfHeaderLines, fieldSeparator, decimalSymbol	Controls reading of the table.
Row selection	idColumnNo/idColumnName and idValue	Optionally filters one entity or stream.
Time selection	timeColumnNo or timeColumnName	Identifies the timestamp column.
Value selection	valueColumnNo or valueColumnName	Identifies the observation column.
Typing	valueType	Selects int, double, string, uri, or bool.

A JSON Schema validator cannot prove that a file exists, that a named column is present, or that number and name selectors identify the same column. These checks require opening the external resource and may depend on network access.

14. SensorConnection mapping

SensorConnection represents how observations can be obtained from a service or endpoint. It is embedded in +Dynamizer because it is connection configuration for that association, not a replacement for the physical sensor, observation, or location entities managed by the external service.

Member	Purpose
connectionType	Service or protocol identifier; CodeValue preserves optional codeSpace.
observationProperty	Phenomenon supplied by the connection.
uom	Unit reported by the stream.
sensorID / sensorName	Service-side sensor identification.
observationID / datastreamID	Service-side observation or datastream identifiers.
baseURL	Base endpoint for the service.
mqttServer / mqttTopic	MQTT connection metadata when applicable.
linkToObservation	Direct observations endpoint.
linkToSensorDescription	Sensor metadata endpoint.
sensorLocation	ID of an associated CityObject.
authType / authConfig	Authentication method and non-secret configuration.

Note: Passwords, access tokens, API keys, and other confidential information must not be stored in a public CityJSON file. The SensorConnection should contain only the non-sensitive information needed to describe the connection, such as the service address, connection type, and sensor or datastream identifier. If authentication is required, the application accessing the sensor service must obtain and manage the credentials separately.

15. Code values, units, and supporting GML mechanisms

15.1. CodeValue

gml:CodeType contains a code and may carry codeSpace. The extension uses a compact string when no vocabulary identifier is required and an expanded object when codeSpace must be preserved.

```
"TimeseriesML"
{ "code": "TimeseriesML", "codeSpace": "https://www.ogc.org/standard/timeseriesml/" }
```

15.2. Units of measure

CityJSON does not impose a native unit model on attributes. The extension therefore carries uom explicitly where required by Dynamizer semantics. A string such as 'Cel' can follow a declared vocabulary, but the extension schema cannot prove dimensional compatibility or perform conversion.

15.3. Boundaries

Mechanism	Representation	Boundary
Code lists	CodeValue	Membership and vocabulary availability are external checks.
Units	uom string	No automatic dimensional analysis or conversion.
External resources	Role-specific URI-reference members	No generic GML ExternalReference implementation.
Object relations	target, dynamicData, timeseries, sensorLocation IDs	No generic CityObjectRelation class.
Feature lifespan	creationDate, terminationDate, validFrom, validTo	Not the CityGML Versioning module.

16. Naming rules and references between CityObjects

Location	Naming rule	Example
New extension CityObject type	Begins with + and an uppercase letter	+Dynamizer
Attribute added to a core CityObject	Begins with +	+dynamizer
Attribute owned by an extension CityObject	No + required	target

Embedded definition member	No + required	timestamp

Every ID-valued relation is represented as a JSON string because CityJSON object identity is expressed by keys in the CityObjects map. Relationships between CityObjects are represented using their IDs. The JSON Schema can verify that dynamicData contains a string, but it cannot confirm that series-temperature actually exists in CityObjects. Therefore, an additional validation step is required to check references such as:

- target;
- dynamicData;
- timeseries;
- sensorLocation;
- +dynamizer.

17. Validation and conformance model

Layer	What it can establish	Typical implementation
1. JSON syntax	Well-formed JSON and parsable values.	JSON parser.
2. Structural/schema	Allowed types, required members, patterns, and typed-pair branches.	cjvalext, cjval, Draft-07 validator.
3. Semantic graph	ID/pointer resolution, target type, time order, inverse consistency, and cycles.	Semantic checker; validate_references.py covers selected rules.
4. External resources	File existence/columns, XML content, service responses, code lists, and units.	Resource-aware application, possibly with network access.

17.1. Required semantic checks

- Resolve target and evaluate attributeRef relative to the target CityObject.
- Resolve dynamicData to one of the four concrete extension series types.
- Verify every target-side +dynamizer entry points back to the same target.
- Verify sensorLocation and component timeseries IDs when present.
- Check startTime <= endTime and firstTimestamp <= lastTimestamp when positions are comparable.
- Require chronological, non-duplicate GenericTimeseries timestamps under the chosen profile.
- Check value compatibility with the target property and the declared valueType.
- Detect CompositeTimeseries cycles and apply a defined overlap/gap policy.
- Validate external file structure and service access according to deployment policy.

Successful schema validation proves structural conformance. It does not prove that references resolve, a sensor endpoint is online, a CSV column exists, units are compatible, or the time-series graph is meaningful.

18. Complex dynamic values: Current limitations

CityGML permits geometry, implicit-geometry, and appearance values in a time series. CityJSON stores geometry in a CityObject's geometry member, templates at geometry-templates, and materials/textures at the root appearance object with index-based assignments. Placing complete native geometry or appearance resources inside TimeValuePair attributes would violate or duplicate those core structures.

A stable and interoperable CityJSON representation for changes to geometry, implicit geometry, and appearance has not yet been established. These values depend on CityJSON structures such as shared vertices, geometry templates, materials, textures, themes, and array-based references. Their representation therefore requires additional design and validation rules that are outside the current scope of the extension. For this reason, version 2.0.0 does not include these dynamic value types.

CityGML value	Why the scalar pattern is insufficient	Version 2.0.0 decision
geometryValue	CityJSON geometry uses shared vertices and native geometry arrays.	Excluded; investigate stable state references separately.
implicitGeometryValue	GeometryInstance depends on root geometry-templates.	Excluded; requires template-aware packaging.
appearanceValue	Materials/textures and their assignments use root arrays and themes.	Excluded; requires unambiguous resource and assignment semantics.

19. CityJSONSeq and streaming considerations

CityJSONSeq serialises CityJSONFeature objects independently. target, dynamicData, and component timeseries are ordinary ID strings, so a generic splitter may place the target, Dynamizer, and series in different features. A consumer reading only one feature can then see unresolved references even though the original CityJSON document was valid.

The core extension remains applicable to ordinary CityJSON. A future CityJSONSeq profile should either require co-location of the target, its Dynamizers, and referenced series in one feature, or define collection-level reference resolution. This is a packaging and processing rule rather than a change to the scalar data model.

20. Conformance classes and claims

Class	Scope
Extension syntax	The extension document conforms to the CityJSON 2.0 Extension meta-schema.
Scalar Dynamizer core	+Dynamizer, five scalar value branches, four concrete series, and SensorConnection.
Semantic reference integrity	Target, pointer, series, inverse, sensor-location, and component reference resolution.
Resource integrity	File and service resources can be accessed and interpreted as declared.
Application profile	Project-specific units, code lists, source requirements, interpolation, and packaging.

Note: The extension provides a practical CityJSON encoding of the supported scalar and operational core of CityGML 3.0 Dynamizer. It does not claim complete support for all CityGML Dynamizer value branches or generic GML mechanisms.

21. Repository examples and scripts

Example	Encoding demonstrated	Where values reside
01 Generic	GenericTimeseries	Embedded typed time-value pairs.
02 Tabulated file	TabulatedFileTimeseries	Bundled CSV file.
03 Standard file	StandardFileTimeseries	Bundled TimeseriesML file.
04 Composite	CompositeTimeseries	Referenced component series.
05 Sensor connection	SensorConnection	TUM FROST service.

Note: All examples use the same TUM LoD2 building and target /attributes/temperature. This isolates the effect of the acquisition or storage pattern. The scripts are non-normative: apply_frost_temperature.py demonstrates enrichment, build_tum_temperature_examples.py regenerates the comparable examples, and validate_references.py checks selected graph and resource rules that JSON Schema cannot express.

22. Known limitations and future work

Area	Current limitation	Recommended future work
Complex values	Geometry, implicit geometry, and appearance are excluded.	Investigate a CityJSON-compatible method for representing these changes.
Semantic validation	Not all rules are executable in JSON Schema.	Publish a formal semantic conformance specification and test suite.
Units and code lists	Strings can be structurally valid but semantically incompatible.	Recommend vocabularies and resolution policies by application profile.

External resources	Validation may require files, network, or credentials.	Separate deterministic checks from resource-dependent checks.
CityJSONSeq	Feature splitting can break ordinary ID links.	Define co-location or collection-level resolution rules.
Conversion from and to CityGML	Some XML-specific information and the currently unsupported dynamic values may not be fully preserved during conversion.	Develop conversion tools that clearly report which information was preserved, adapted, or not converted.
Viewer behaviour	Ordinary viewers do not automatically animate values.	Define a consumer API and evaluate viewer/database implementations.

23. Implementation checklist

- Add +dynamizer to the target CityObject when a reverse link to its Dynamizers is needed.
- Create a +Dynamizer containing target and an attributeRef evaluated inside that target.
- Provide the changing values through one of the four supported time-series CityObjects or through an embedded SensorConnection.
- Use RFC 3339 date-time strings with a UTC marker or numerical offset for exact timestamps.
- Use the expanded TimePosition object only when reduced precision or additional temporal information must be preserved.
- Ensure that valueType matches the value member used in every TimeValuePair.
- Validate the JSON structure, extension schema, references between objects, and external resources through the appropriate validation steps.
- Do not store passwords, tokens, API keys, or other confidential information in the CityJSON document.
- During conversion, clearly report geometry, implicit-geometry, and appearance changes that are not supported by extension version 2.0.0.

24. References

- OGC CityGML 3.0 Part 1: Conceptual Model — https://docs.ogc.org/is/20-010/20-010.html
- OGC CityGML 3.0 Part 2: GML Encoding — https://docs.ogc.org/is/21-006r2/21-006r2.html
- CityGML 3.0 Dynamizer XSD documentation — https://opengeospatial.github.io/CityGML-3.0Encodings/xsd-doc/3.0/dynamizer/
- CityJSON 2.0.2 specification — https://www.cityjson.org/specs/2.0.2/
- CityJSON Extension mechanism — https://www.cityjson.org/specs/2.0.2/#extensions
- CityJSON mapping to CityGML 3.0 — https://www.cityjson.org/citygml/v30/
- RFC 6901: JSON Pointer — https://www.rfc-editor.org/rfc/rfc6901
- RFC 3339: Date and Time on the Internet — https://www.rfc-editor.org/rfc/rfc3339
- W3C XML Schema Part 2: Datatypes, duration — https://www.w3.org/TR/xmlschema-2/#duration

- OGC TimeseriesML 1.2 — https://docs.ogc.org/is/15-042r5/15-042r5.html
- OGC SensorThings API — https://www.ogc.org/standard/sensorthings/
- CityJSON Noise Extension tutorial — https://www.cityjson.org/tutorials/extension/
- CityJSON Energy Extension precedent - https://github.com/ozgetufan/cjenergy/tree/master/schemas/extensions
- Earlier CityJSON Dynamizer proposal - https://github.com/1khawla/CityJSON-Dynamizer